Quantile Mapping Bias Correction

Quick dive with python code

6 min read · Feb 22, 2023

Miguel Gutierrez Follow

Listen Share

Introduction

Quantile mapping is a powerful technique for correcting biases in projection models by aligning the observed quantile distributions with the model projections. This method is widely used in climate models, but can also be applied to other areas where decision-making is based on distributional projections, such as supply chain models. What makes quantile mapping particularly attractive is its simplicity and

computational efficiency compared to other bias correction methods like *EMOS* and *Double-Q-Learning* (Reinforcement learning).

In this example, we will use Python code to extend the case study created by Chongua Yin (Explain Quantile Mapping Bias Correction with Python code) , thanks Chongua :). We will demonstrate the effectiveness of quantile mapping in correcting systematic biases in quantile forecasts/projections as the code of how the implementation works.

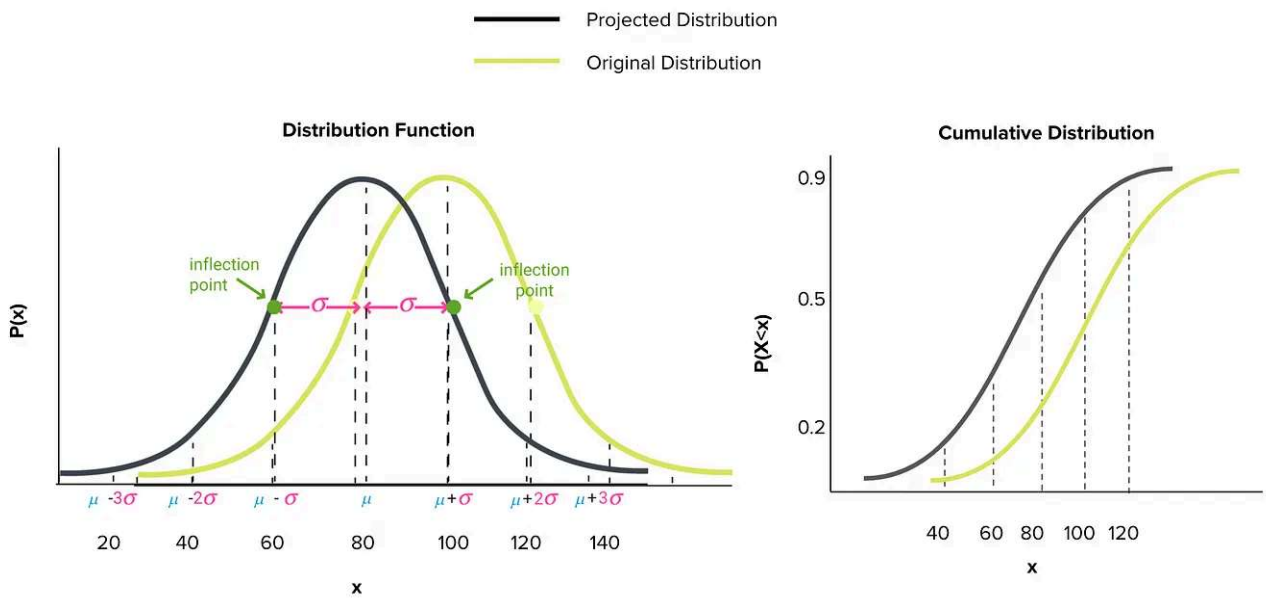
Understanding the idea lets you understand easier the new advances on this topic and **most important** implement it based on the necessities of your project.

Variables and setting the enviroment

In this case study, we will demonstrate the use of quantile mapping to correct a systematic bias in a quantile forecast/projection. We will work with four datasets:

- The observed past data *(ref_dataset),*
- The observed future data *(ref_dataset_future),*
- The model simulation/forecast during the same period as the observation past data *(model_present).*
- The model projection/forecast for the future period *(model_future).*

Assuming that the observational data *(ref_dataset)* follows a Normal distribution with mean 100 and standard deviation 20, and that the future projection *(ref_dataset_future)* is the same distribution shifted to the right with mean 100 + 2, we will create the model projection/forecast (*model_present* and *model_future*) with a systematic bias of 20 units downwards. This means that the quantile forecast/projection will be underestimated by 20 units, made on purpose.



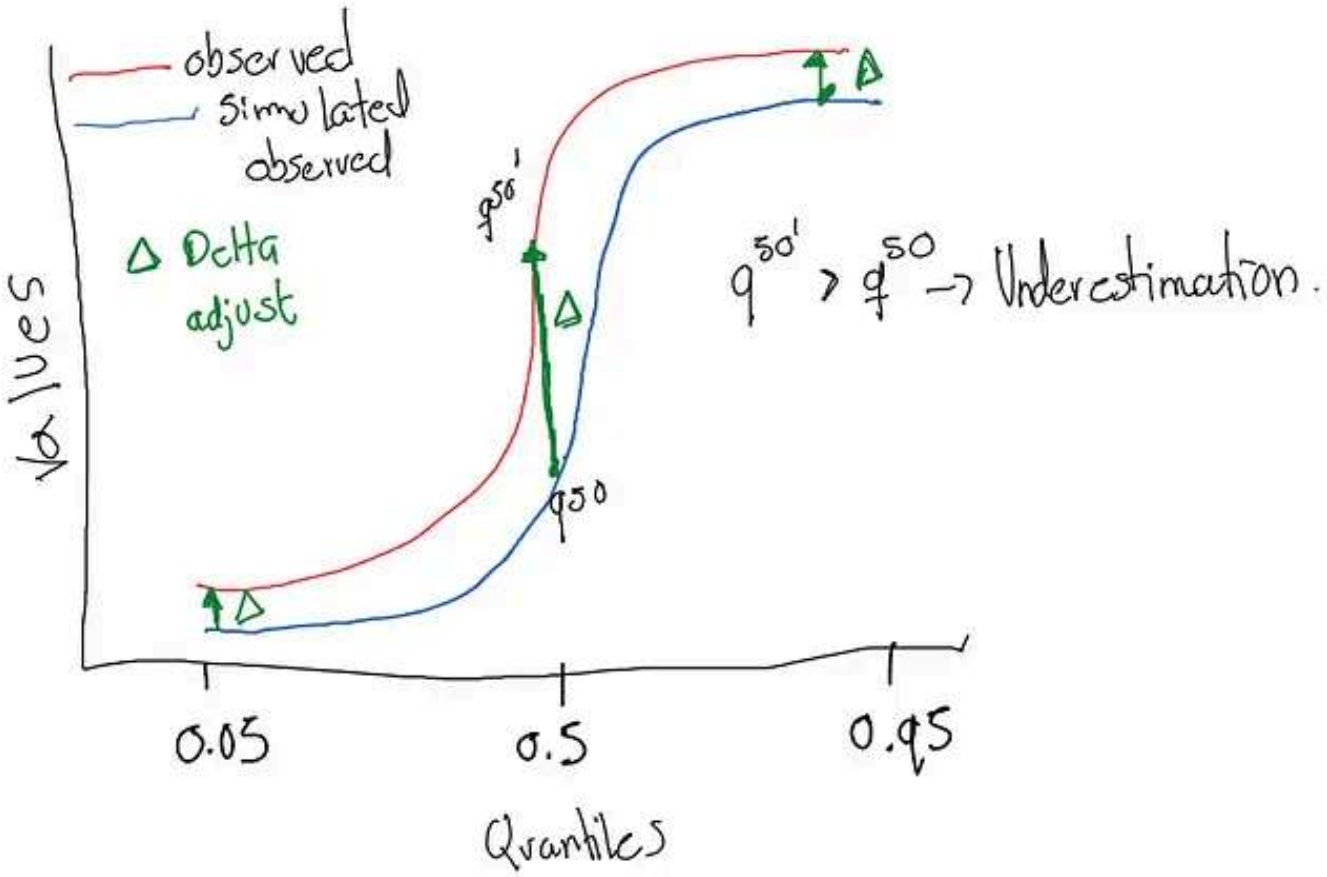

```
import numpy as np
from scipy.stats import norm, gamma, erlang, expon


# Data
u1, s1 = 100,20
bias = 20
bias_d = 2
n = 100
q_ = 20
dist1 = norm(loc=u1, scale=s1)
dist2 = norm(loc=u1+bias_d, scale=s1)

quantiles = [round(x*0.05,2) for x in range(1,q_)]

q_dist1 = [dist1.ppf(x*0.05) for x in range(1,q_)]
q_dist2 = [dist2.ppf(x*0.05) for x in range(1,q_)]

# Distribution real sample
ref_dataset = np.random.normal(u1,s1,n).round(2)
# Sub-estimated
model_present = (q_dist1 - np.random.normal(bias,2,q_-1)).round(2)
# Future model with the same bias
model_future = (q_dist2 - np.random.normal(bias,2,q_-1)).round(2)
```

How Quantile mapping work?

Quantile mapping works by removing the systematic bias between the observed data and the forecast/projection quantile distributions. The idea behind this

technique is to calibrate the forecast/projection distribution with the observed distribution by adjusting the forecast/projection values.

To visualize this concept, consider the following graph. The red line represents the observed data quantile function, while the blue line represents the forecast/simulated distribution with a systematic bias. The objective of quantile mapping is to move the forecast/projection values upwards to match the observed values. The resulting change shown in green, represents the correction between the forecast/projection distribution and observer data.



In essence, quantile mapping provides a way to adjust the forecast/projection distribution to better align with the observed data distribution, thereby removing any systematic biases. This results in improved accuracy and more reliable decision-making based on the corrected forecast/projection distributions.

In this case study, we will apply the eQM Delta-proportion method of quantile mapping. The eQM Delta-proportion method assumes that the proportional difference between the downscaled and observed value during the period of observed data applies as a systematic bias to the future period as well.

We will correct both the present observed data and the future observed data, allowing us to test the effectiveness of the method on both the “training data” and the “testing data”. By doing so, we can ensure that the correction method works well on both known and unknown data and provides reliable results.

For a given percentile, the eQM Delta-proportion method calculates the proportional difference between the downscaled and observed values during the observed data period. This difference is then applied to the future period as a correction factor, effectively removing the systematic bias in the forecast/projection distribution. The corrected forecast/projection distribution, aligned with the observed data distribution, will provide more accurate and reliable results for decision-making.

```
def eQM_porcentual_delta(ref_dataset, model_present, model_future):
        """
        Remove the biases for each quantile value taking the difference between
        ref_dataset and model_present at each percentile as a kind of systemati
        and add them to model_future at the same percentile.

        returns: downscaled model_present and model_future
        """

        model_present_corrected = np.zeros(model_present.size)
        model_future_corrected = np.zeros(model_future.size)

        for ival, model_value in enumerate(model_present):
            percentile = percentileofscore(model_present, model_value)
            percentile_ref = np.percentile(ref_dataset, percentile)
            dif = (percentile_ref - model_value)/model_value
            model_present_corrected[ival] = model_value*(1+dif)
            model_future_corrected[ival] = model_future[ival]*(1+dif)
```

```
        return model_present_corrected, model_future_corrected
```

Let's see how it works

We apply the corresponding correction

```
model_present, model_future = eQM_porcentual_delta(ref_dataset, model_present,
```

We check that the systematic correction works fine. For this, we check the quantile calibration (quantile-quantile map) now matches the observed data. The idea is that corrected quantiles must be closer to theoretical quantiles.

```
list_qp = []
list_qp_corrected = []

for i, q in enumerate(model_present):
    count = 0
    count_corrected = 0
    for j in ref_dataset:
        q_p = model_present[i]
        q_p_corrected = model_present_corrected[i]
        if j<=q_p:
            count +=1
        if j<=q_p_corrected:
            count_corrected +=1
    perc_qp = count/n
    list_qp.append(perc_qp)

    perc_qp_corrected = count_corrected/n
    list_qp_corrected.append(perc_qp_corrected)

dif = np.array(quantiles)-(list_qp)
dif = np.mean(dif[dif<=0]), np.mean(dif[dif>0])
dif_a = np.mean(np.array(quantiles)-(list_qp))
dif_a = np.mean(dif_a[dif_a<=0]), np.mean(dif_a[dif_a>0])

dif = np.array(quantiles)-(list_qp_corrected)
dif = np.mean(dif[dif<=0]), np.mean(dif[dif>0])
dif_corrected = np.mean(np.array(quantiles)-(list_qp_corrected))
dif_corrected = np.mean(dif_corrected[dif_corrected<=0]), np.mean(dif_corrected
```

```
print("EQM Delta proportion")
print("Previous Difference", dif_a)
print("Before    ", list_qp)
print("Teorical Quantile  ", quantiles)
print("Corrected",list_qp_corrected)
print("Difference with correction",dif)
```

```
EQM Delta proportion
Previous Difference (nan, 0.25736842105263164)
Before     [0.0, 0.0, 0.02, 0.01, 0.04, 0.09, 0.07, 0.14, 0.14, 0.14, 0.14, 0.22, 0.31, 0.33, 0.42, 0.5, 0.62, 0.67, 0.75]
Teorical Quantile   [0.05, 0.1, 0.15, 0.2, 0.25, 0.3, 0.35, 0.4, 0.45, 0.5, 0.55, 0.6, 0.65, 0.7, 0.75, 0.8, 0.85, 0.9, 0.95]
Corrected [0.06, 0.11, 0.21, 0.16, 0.27, 0.37, 0.32, 0.42, 0.58, 0.47, 0.53, 0.63, 0.68, 0.73, 0.79, 0.84, 0.89, 0.94, 1.0]
Difference with correction (-0.04133333333333333, 0.030000000000000006)
```

This image is from a run example. We can see that the quantile bias downwards of 25% was reduced to a 3% downward bias and a 4% upper bias. This is we pass a 28% bias to a 7% absolute bias, this is we reduce the bias a 21%. Of course, this is with the previously observed data, what about the future projection/forecast?

```
list_qpf = []
list_qpf_corrected = []

for i, q in enumerate(model_future):
    count = 0
    count_corrected = 0
    for j in ref_dataset:
        q_f = model_future[i]
        q_f2 = model_future_corrected[i]
        if j<=q_f:
            count +=1
        if j<=q_f2:
            count_corrected +=1

    perc_qpf = count/n
    list_qpf.append(perc_qpf)

    perc_qpf_corrected = count_corrected/n
    list_qpf_corrected.append(perc_qpf_corrected)

dif = np.array(quantiles)-(list_qpf)
dif = np.mean(dif[dif<=0]), np.mean(dif[dif>0])
dif_a = np.mean(np.array(quantiles)-(list_qpf))
dif_a = np.mean(dif_a[dif_a<=0]), np.mean(dif_a[dif_a>0])

dif = np.array(quantiles)-(list_qpf_corrected)
dif = np.mean(dif[dif<=0]), np.mean(dif[dif>0])
dif_corrected = np.mean(np.array(quantiles)-(list_qpf_corrected))
```

```
dif_corrected = np.mean(dif_corrected[dif_corrected<=0]), np.mean(dif_corrected

print("EQM Delta proportion in FUTURE DATA")
print("Previous Difference", dif_a)
print("Before    ", list_qp)
print("Teorical Quantile  ", quantiles)
print("Corrected",list_qp_corrected)
print("Difference with correction",dif)
```

```
EQM Delta proportion in FUTURE DATA
Previous Difference (nan, 0.23578947368421055)
```


With this correction, we can see we reduce the bias in “test data” of about 13%. Keep in mind that quantile mapping assumes a time-invariant transfer function, therefore is better to make corrections with sufficient data close to the future projection date.

Conclusion

The results of this case study suggest that bias correction effectively reduced the downward bias. While more advanced and sophisticated solutions exist, as noted in the references, their code implementations and ideas can be difficult to locate. Nevertheless, the use of bias correction demonstrated in this study provides a simple and effective means to address the issue. I will add in the references some good papers and blogs which talks other topics of calibration.

References

Yin, C. (2019). Explain Quantile Mapping Bias Correction in Python with Code. LinkedIn. https://www.linkedin.com/pulse/explain-quantile-mapping-bias-correction-python-code-chonghua-yin/.

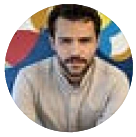
Maraun, D. (2013). Bias Correction, Quantile Mapping, and Downscaling: Revisiting the Inflation Issue. *Journal of Climate*, *26*(6), 2137–2143. http://www.jstor.org/stable/26192268

Kakimoto, M., Shiga, Y., Shin, H., Ikeda, R., & Kusaka, H. (2022). Quantile mapping correction of analog ensemble forecast for solar irradiance. Solar Energy, 237, 253–263. https://doi.org/10.1016/j.solener.2022.03.015

Wang, QJ (2017). How suitable is quantile mapping for post-processing GCM precipitation forecasts? Hepex. https://hepex.inrae.fr/how-suitable-is-quantile-mapping-for-post-processing-gcm-precipitation-forecasts/.

Noarov, G. and Roth, A. (2023) “The Scope of Multicalibration: Characterizing Multicalibration via Property Elicitation”, Arxiv. doi:10.48550/arXiv.2302.08507.

Quantile Mapping | Bias Correction | Python | Probabilistic Forecast | Forecast



Follow

Written by Miguel Gutierrez

35 followers · 6 following

Msc Statistical Learning At Politecnico di Milano | Data Scientist and AI Engineerg

Responses (1)

Write a response

What are your thoughts?

sahar

Nov 15, 2023

Hello, I did an exponential microscale project related to the earth's temperature and I want to do bais correction on my data. I have two inputs here, one is observational data and one is exponential downscaling data, but you have two other inputs... more


1 reply Reply